

or what we would usually write as 4. If we write 1001, we interpret it in binary as

$$1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

or what we would usually write as 9. After running this mini-algorithm to get the binary expansion of 89, we are poised to easily run the full algorithm and complete the multiplication process.

Implementing RPM in Python

It's relatively simple to implement RPM in Python. Let's say that we want to multiply two numbers that we will call n_1 and n_2 . First, let's open a Python script and define these variables:

```
n1 = 89
n2 = 18
```

Next, we'll start our halving column. Just as described, the halving column begins with one of the numbers we want to multiply:

```
halving = [n1]
```

The next entry will be `halving[0]/2`, ignoring the remainder. In Python, we can use the `math.floor()` function to accomplish this. This function just takes the closest integer less than a given number. For example, the second row of the halving column can be calculated as follows:

```
import math
print(math.floor(halving[0]/2))
```

If you run this in Python, you'll see that the answer is 44.

We can loop through each row of the halving column, and in each iteration of our loop, we will find the next entry in the halving column in the same way, stopping when we reach 1:

```
while(min(halving) > 1):
    halving.append(math.floor(min(halving)/2))
```

This loop uses the `append()` method for concatenation. At each iteration of the `while` loop, it concatenates the `halving` vector with half of its last value,